

AI Assisted Coding

Assignment – 11.3

Name: K. Sai Krishna Reddy

Roll No:2303A52305

Batch:45

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Scenario

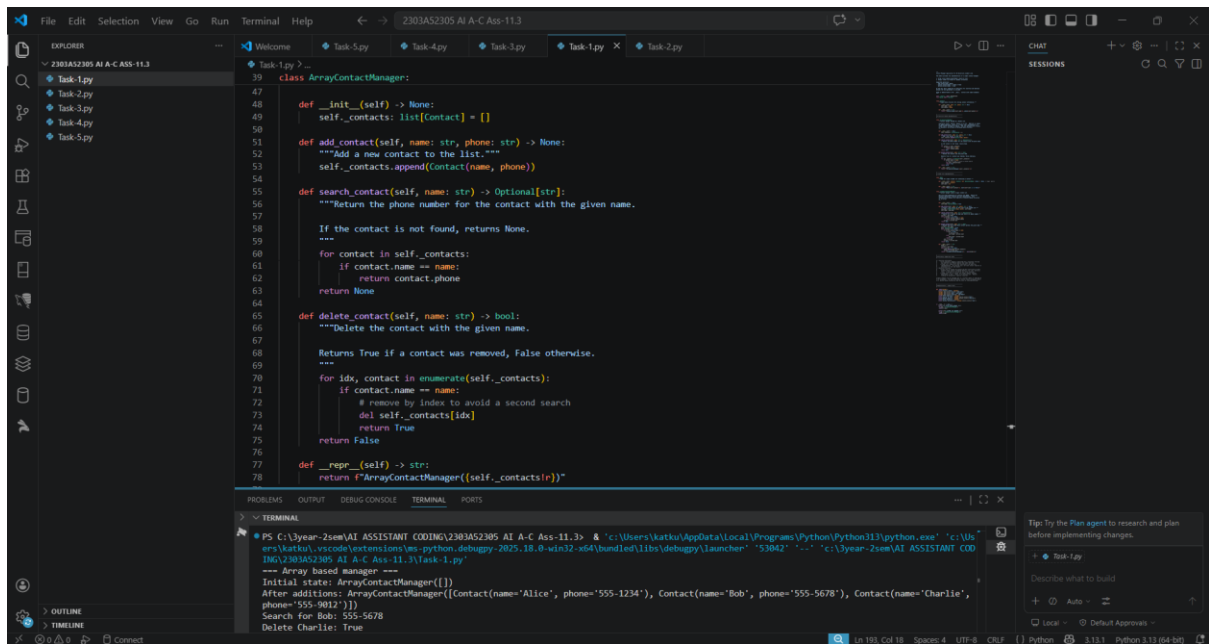
SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

Tasks

1. Implement the contact manager using arrays (lists).
2. Implement the same functionality using a linked list for dynamic memory allocation.
3. Implement the following operations in both approaches:
 - o Add a contact
 - o Search for a contact
 - o Delete a contact
4. Use GitHub Copilot to assist in generating search and delete methods.
5. Compare array vs. linked list approaches with respect to:
 - o Insertion efficiency
 - o Deletion efficiency

Expected Outcome

- Two working implementations (array-based and linked-list-based).
- A brief comparison explaining performance differences



Task 2: Library Book Search System (Queues & Priority Queues)

Scenario

The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.

Tasks

1. Implement a Queue (FIFO) to manage book requests.
2. Extend the system to a Priority Queue, prioritizing faculty requests.
3. Use GitHub Copilot to assist in generating:

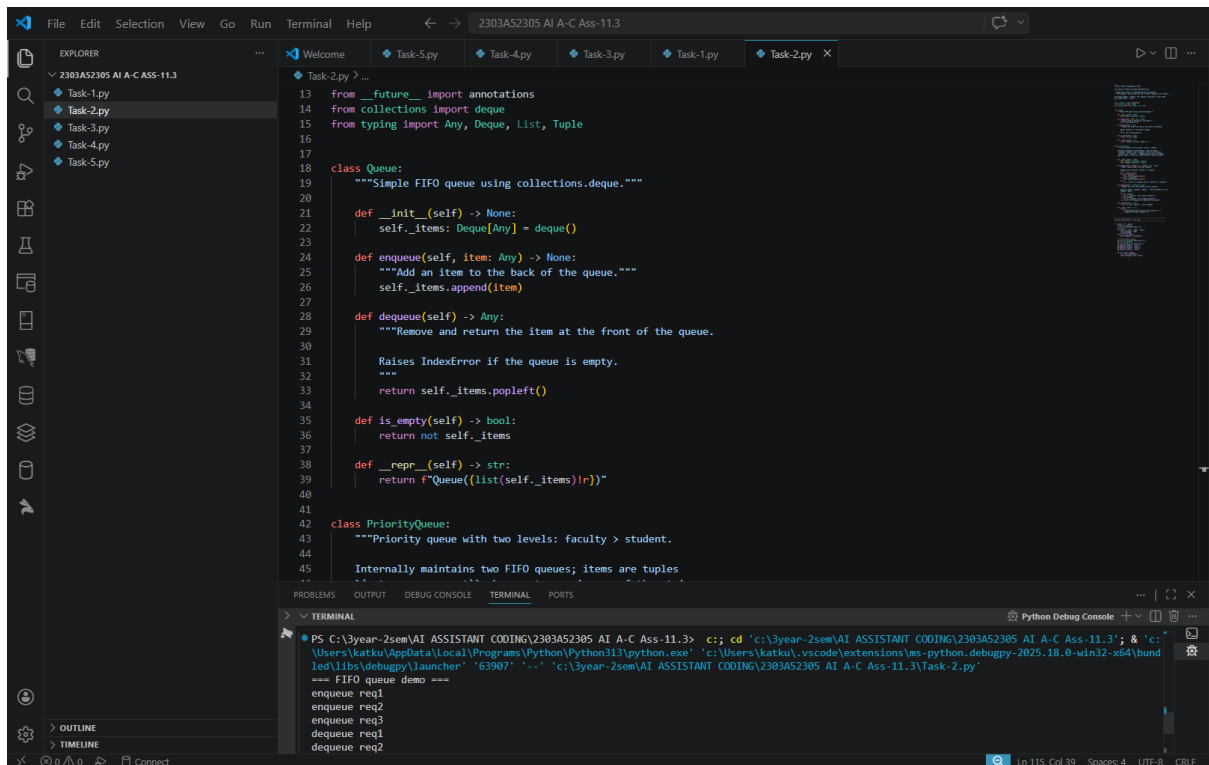
o enqueue() method

o dequeue() method

4. Test the system with a mix of student and faculty requests.

Expected Outcome

- Working queue and priority queue implementations.
- Correct prioritization of faculty requests.



```
13 from __future__ import annotations
14 from collections import deque
15 from typing import Any, Deque, List, Tuple
16
17
18
19 class Queue:
20     """Simple FIFO queue using collections.deque."""
21
22     def __init__(self) -> None:
23         self._items: Deque[Any] = deque()
24
25     def enqueue(self, item: Any) -> None:
26         """Add an item to the back of the queue."""
27         self._items.append(item)
28
29     def dequeue(self) -> Any:
30         """Remove and return the item at the front of the queue.
31         Raises IndexError if the queue is empty.
32         """
33         return self._items.popleft()
34
35     def is_empty(self) -> bool:
36         return not self._items
37
38     def __repr__(self) -> str:
39         return f"Queue({list(self._items)})"
40
41
42 class PriorityQueue:
43     """Priority queue with two levels: faculty > student.
44     Internally maintains two FIFO queues; items are tuples
45     """
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-11.3> c:\cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-11.3'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '63907' '-' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-11.3\Task-2.py'
=== FIFO queue demo ===
enqueue req1
enqueue req2
enqueue req3
dequeue req1
dequeue req2
```

Task 3: Emergency Help Desk (Stack Implementation)

Scenario

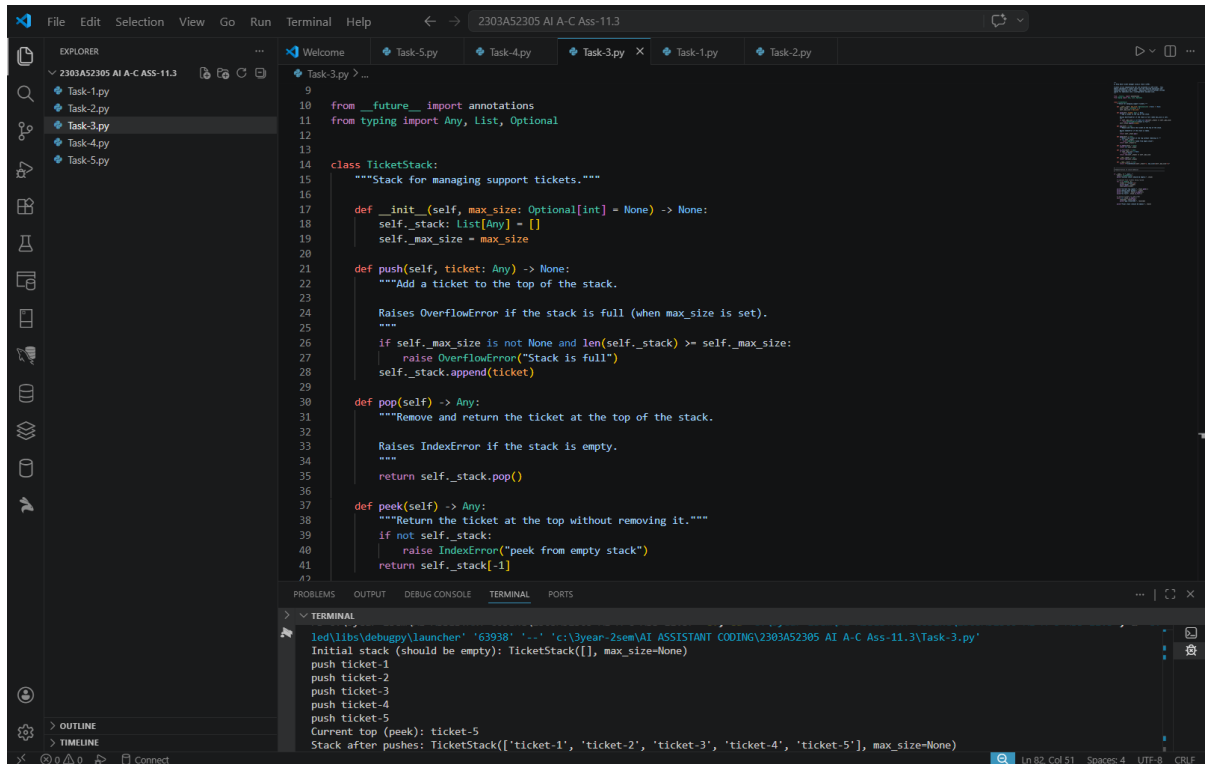
SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

Tasks

1. Implement a Stack to manage support tickets.
2. Provide the following operations:
 - o push(ticket)
 - o pop()
 - o peek()
3. Simulate at least five tickets being raised and resolved.
4. Use GitHub Copilot to suggest additional stack operations such as:
 - o Checking whether the stack is empty
 - o Checking whether the stack is full (if applicable)

Expected Outcome

- Functional stack-based ticket management system.
- Clear demonstration of LIFO behavior.



The screenshot shows a VS Code editor with a Python file named `Task-3.py`. The code defines a `TicketStack` class with methods `__init__`, `push`, `pop`, and `peek`. The `push` method checks for a maximum size and raises `OverflowError` if the stack is full. The `pop` method raises `IndexError` if the stack is empty. The `peek` method also raises `IndexError` if the stack is empty. The terminal output shows the execution of the code, demonstrating the LIFO behavior of the stack.

```
9
10 from __future__ import annotations
11 from typing import Any, List, Optional
12
13
14 class TicketStack:
15     """Stack for managing support tickets."""
16
17     def __init__(self, max_size: Optional[int] = None) -> None:
18         self._stack: List[Any] = []
19         self._max_size = max_size
20
21     def push(self, ticket: Any) -> None:
22         """Add a ticket to the top of the stack.
23
24         Raises OverflowError if the stack is full (when max_size is set).
25         """
26         if self._max_size is not None and len(self._stack) >= self._max_size:
27             raise OverflowError("Stack is full")
28         self._stack.append(ticket)
29
30     def pop(self) -> Any:
31         """Remove and return the ticket at the top of the stack.
32
33         Raises IndexError if the stack is empty.
34         """
35         return self._stack.pop()
36
37     def peek(self) -> Any:
38         """Return the ticket at the top without removing it."""
39         if not self._stack:
40             raise IndexError("peek from empty stack")
41         return self._stack[-1]
```

Terminal Output:

```
led\libs\debuggy\launcher" "63938" '-' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-11.3\Task-3.py'
Initial stack (should be empty): TicketStack([], max_size=None)
push ticket-1
push ticket-2
push ticket-3
push ticket-4
push ticket-5
Current top (peek): ticket-5
Stack after pushes: TicketStack(['ticket-1', 'ticket-2', 'ticket-3', 'ticket-4', 'ticket-5'], max_size=None)
```

Task 4: Hash Table

Objective

To implement a Hash Table and understand collision handling.

Task Description

Use AI to generate a hash table with:

- Insert
- Search
- Delete

Starter Code

class HashTable:

pass

Expected Outcome

- Collision handling using chaining
- Well-commented methods

The screenshot shows a VS Code editor with a Python file named `Task-4.py`. The code implements a HashTable using separate chaining for collision handling. It includes methods for inserting, searching, updating, and deleting items. The terminal output shows the execution of the script, demonstrating the initial state, insertions of 'apple', 'banana', 'grape', and 'cherry', and subsequent searches and deletions.

```

74 if __name__ == "__main__":
75     table = HashTable(capacity=4) # small to force collisions
76     print("Initial table:", table)
77
78     # insert several items, including ones that collide by bucket
79     pairs = [("apple", 1), ("banana", 2), ("grape", 3), ("cherry", 4)]
80     for k, v in pairs:
81         print(f"insert {k} -> {v}")
82         table.insert(k, v)
83
84     print("Table after inserts:", table)
85
86     # search existing and non-existing
87     print("search apple:", table.search("apple"))
88     try:
89         print("search orange:", table.search("orange"))
90     except KeyError:
91         print("orange not found")
92
93     # update a value
94     table.insert("banana", 20)
95     print("banana updated:", table.search("banana"))
96
97     # delete a key
98     table.delete("grape")
99     print("after deleting grape:", table)
100
101     # attempt to delete missing key
102     try:
103         table.delete("pear")
104     except KeyError:
105         print("pear not found, cannot delete")

```

```

PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-11.3>
c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-11.3> cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-11.3' & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50599' '-...' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-11.3\Task-4.py'
Initial table: HashTable(capacity=4, buckets=[[], [], [], []])
insert apple -> 1
insert banana -> 2
insert grape -> 3
insert cherry -> 4
Table after inserts: HashTable(capacity=4, buckets=[(['grape', 3],), [], [('apple', 1), ('cherry', 4)], [('banana', 2)]])

```

Task 5: Real-Time Application Challenge

Scenario

Design a Campus Resource Management System with the following features:

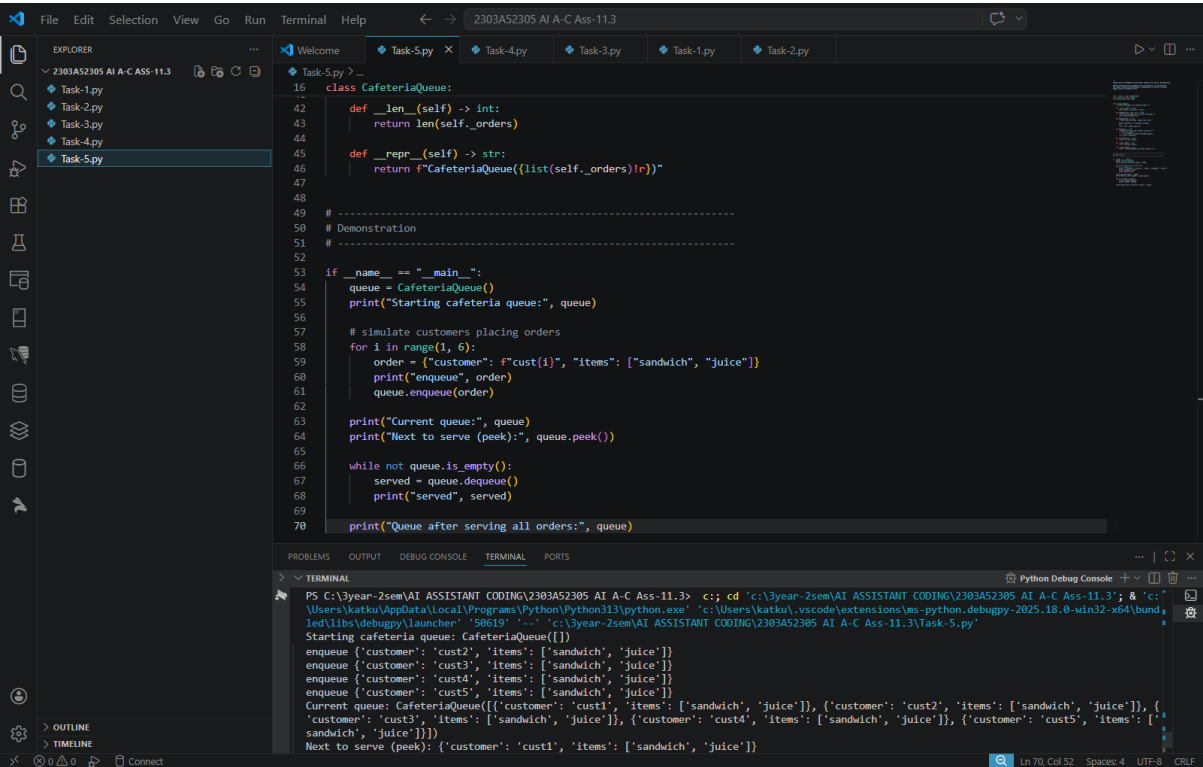
- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

Student Tasks

1. Choose the most appropriate data structure for each feature.
2. Justify your choice in 2–3 sentences.
3. Implement one selected feature using AI-assisted code generation.

Expected Outcome

- Mapping table: Feature → Data Structure → Justification
- One fully working Python implementation



Feature → Data Structure → Justification

| Feature | Data Structure | Justification |

| Student Attendance Tracking | Set or dictionary | Fast membership checks for present/absent, unique entries; O(1) lookup. |

| Event Registration System | List or set | Preserve order of signup (list) or prevent duplicates (set); simple iteration. |

| Library Book Borrowing | Hash table / dict | Map book IDs to borrower info; O(1) insert/search/delete. |

| Bus Scheduling System | Priority queue | Serve buses by departure time or priority; allows efficient retrieval of next departure. |

| Cafeteria Order Queue | FIFO queue | Orders must be handled in the order they arrive; enqueue/dequeue semantics. |

We will implement the Cafeteria Order Queue below.